

React Native MoveNet - Setup Instructions

☆☆ Features

- 📷 **Camera Integration** - Ambil foto langsung dari kamera mobile
- 🖼️ **Gallery Picker** - Pilih gambar dari galeri
- 🤖 **Pose Detection** - Deteksi pose menggunakan MoveNet model
- 📊 **Real-time Analysis** - Analisis keypoints dan confidence score
- 🎨 **Visual Overlay** - Tampilan skeleton overlay pada hasil deteksi

📦 Installation

1. Install Dependencies

```
cd react-native-movenet
npm install
```

2. Install Additional Packages

```
# Required for path aliases
npm install --save-dev babel-plugin-module-resolver

# For SVG support (skeleton drawing)
npm install react-native-svg
```

3. Setup Model Files

⚠️ **IMPORTANT:** Model harus di-convert ke TFJS format!

See [../MODEL_CONVERSION.md](#) untuk detail lengkap.

Quick steps:

1. Download dari: <https://tfhub.dev/google/tfjs-model/movenet/singlepose/lightning/4>
2. Extract dan copy ke `assets/models/`
3. Verify files: `model.json` dan `*.bin` files

🚀 Running the App

Development Mode

```
# Start Metro bundler
npm start

# Or directly run on platform:
npm run android # Android device/emulator
npm run ios     # iOS simulator (macOS only)
npm run web     # Web browser
```

Using Expo Go

1. Install Expo Go app on your phone:

- Android: <https://play.google.com/store/apps/details?id=host.exp.exponent>
- iOS: <https://apps.apple.com/app/expo-go/id982107779>

2. Run `npm start`

3. Scan QR code dengan Expo Go app

Project Structure

```
react-native-movenet/
├── App.js                # Main app entry
├── babel.config.js       # Babel configuration
├── package.json          # Dependencies
├── app.json              # Expo configuration
├──
├── assets/
│   └── models/           # ⚠ Place converted model here
│       ├── model.json
│       └── *.bin files
├──
├── src/
│   ├── components/
│   │   └── PoseOverlay.js # Draw keypoints & skeleton
│   ├──
│   ├── models/
│   │   ├── MoveNetModel.js # Model wrapper
│   │   └── PoseDetector.js  # High-level API
│   ├──
│   └── utils/
│       ├── constants.js    # Keypoint definitions
│       └── imagePreprocessing.js # Image preprocessing
```

PROF

Configuration

Update Model Path

Edit `src/models/MoveNetModel.js`:

```
// Line ~45-48
const modelJson = require('../assets/models/model.json');
const modelWeights = require('../assets/models/group1-shard1of1.bin');

// Update filename sesuai dengan file .bin yang Anda punya
```

Permissions

Already configured in `app.json`:

- Camera permission
- Storage permission (read/write)

Usage Example

Basic Usage

```
import PoseDetector from './src/models/PoseDetector';
import * as ImagePicker from 'expo-image-picker';
import { decodeJpeg } from '@tensorflow/tfjs-react-native';

// Initialize (do once at app start)
await PoseDetector.initialize();

// Pick image
const result = await ImagePicker.launchImageLibraryAsync({...});
const imageUri = result.assets[0].uri;

// Load image as tensor
const response = await fetch(imageUri, {}, { isBinary: true });
const imageData = await response.arrayBuffer();
const imageTensor = decodeJpeg(new Uint8Array(imageData));

// Detect pose
const poseResult = await PoseDetector.detectPose(imageTensor);

// Use results
console.log('Detected keypoints:', poseResult.keypoints);
console.log('Quality:', poseResult.stats.detectionQuality);

// Don't forget cleanup!
imageTensor.dispose();
```

With Camera (Coming Soon)

```
import { Camera } from 'expo-camera';

// Camera component with real-time detection
// Will be implemented in future version
```

Debugging

Enable Verbose Logging

Edit `src/models/MoveNetModel.js`:

```
// Add at top of file
const DEBUG = true;

// Then add logging throughout:
if (DEBUG) console.log('Debug info:', ...);
```

Check TensorFlow.js Backend

```
import * as tf from '@tensorflow/tfjs';

console.log('Backend:', tf.getBackend());
console.log('Memory:', tf.memory());
```

Performance Profiling

PROF

```
// Wrap detection code
const startTime = Date.now();
const result = await PoseDetector.detectPose(imageTensor);
const endTime = Date.now();
console.log(`Inference time: ${endTime - startTime}ms`);
```

⚡ Performance Tips

1. Reduce Image Size

```
// Before detection, resize image
const resized = tf.image.resizeBilinear(imageTensor, [384, 384]);
```

```
// Note: Still need to resize to 192x192 for model, but starting smaller helps
```

2. Use Web Assembly Backend (Web only)

```
import '@tensorflow/tfjs-backend-wasm';  
await tf.setBackend('wasm');
```

3. Reuse Tensors

```
// DON'T create new tensors in loop  
// DO reuse or use tf.tidy()  
  
tf.tidy(() => {  
  // All tensors created here auto-disposed  
  const result = processImage(tensor);  
  return result.clone(); // Clone what you want to keep  
});
```

4. Batch Processing (Future)

```
// Process multiple images at once  
const batch = tf.stack([img1, img2, img3]);  
const results = await model.predict(batch);
```

Testing

PROF

Test Model Loading

```
# Create test file: __tests__/model.test.js  
npm test
```

Manual Testing Checklist

- ☐ App starts without errors
- ☐ Can pick image from gallery
- ☐ Image displays correctly
- ☐ Detection runs without crashing
- ☐ Keypoints visible on image
- ☐ Console shows detection stats

- ☐ No memory leaks (check after multiple detections)

Build for Production

Android

```
# Create APK
eas build --platform android

# Or use Expo build
expo build:android
```

iOS

```
# Requires Apple Developer account
eas build --platform ios
```

Updates & Maintenance

Update Dependencies

```
npm update
```

Update Expo SDK

```
expo upgrade
```

Clear Cache (if issues)

```
npm start -- --clear
```

Resources

- Expo Docs: <https://docs.expo.dev/>
- TensorFlow.js: <https://www.tensorflow.org/js>
- React Native: <https://reactnative.dev/>
- MoveNet: <https://tfhub.dev/google/movenet>

Common Issues

"Module not found: model.json"

→ Model belum di-convert/copy. See MODEL_CONVERSION.md

"Backend not initialized"

→ Add `await tf.ready()` before using model

"Out of memory"

→ Dispose tensors: `tensor.dispose()` atau gunakan `tf.tidy()`

"Camera not working"

→ Check permissions di app.json dan device settings

Customization

Change Skeleton Colors

Edit `src/utils/constants.js`:

```
export const KEYPOINT_COLORS = {  
  face: '#YOUR_COLOR',  
  leftArm: '#YOUR_COLOR',  
  // ...  
};
```

— Adjust Confidence Threshold

PROF

```
const result = await PoseDetector.detectPose(imageTensor, {  
  confidenceThreshold: 0.5 // Default: 0.3  
});
```

Hide/Show Labels

```
<PoseOverlay  
  keypoints={poseResult.keypoints}  
  showLabels={true} // Show confidence labels  
  // ...  
>
```

